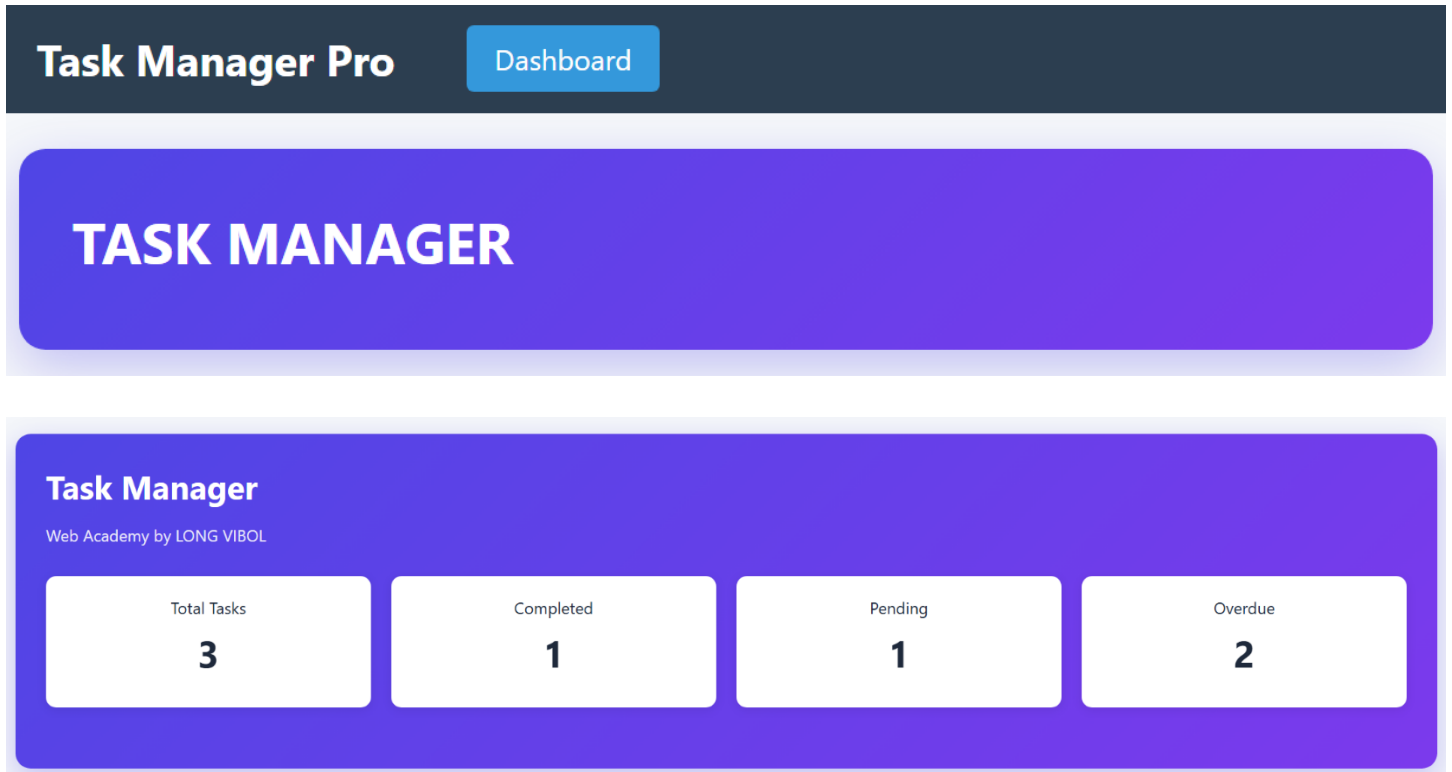


Task Manager App with Angular

```
task-manager.html x TS task-manager.ts
src > app > task-manager > task-manager.html > div.task-manager-container
Go to component
1 <div class="task-manager-container">
2 <!-- header section starts -->
3 <div class="header-section">
4
5 <h1>{{'Task Manager' | uppercase}}</h1>
6
7 </div>
8
9 </div>
```

- យើងដាក់ “នៅក្នុង {{}} interpolation ដើម្បីធ្វើការ ប្រើប្រាស់ syntax from Angular like pipe



- Create card for Total Tasks, Completed, Pending and Overdue

```

<!-- Statistics Cards starts -->
<div class="stats-container">
  <div class="stat-card">
    <div class="stat-label">Total Tasks</div>
    <div class="stat-value">{{tasks.length}}</div>
  </div>

  <div class="stat-card">
    <div class="stat-label">Completed</div>
    <div class="stat-value">{{getCompletedTaskCount()}}</div>
  </div>

  <div class="stat-card">
    <div class="stat-label">Pending</div>
    <div class="stat-value">{{getPendingTaskCount()}}</div>
  </div>

  <div class="stat-card">
    <div class="stat-label">Overdue</div>
    <div class="stat-value">{{getOverdueTasksCount()}}</div>
  </div>
</div>

</div>

<!-- Statistics Cards end -->

```

- Create drop down selection

All Status ▾

All Categories ▾

All Priorities ▾

☒ Show Completed Tasks

- Create alert notification

[ALERT] Time to get back on track! Start with small, easy tasks.

➔ Create condition in the TS and use switch to meet condition in the TS

TS:

```
getProductivityLevel(){  
  const rate = this.getCompletionRate();  
  if (rate >=80) return 'excellent';  
  if (rate >=60) return 'good';  
  if (rate >=40) return 'needs-improvement';  
  return 'poor';  
}
```

HTML

```
<!-- Productivity alert start -->  
<div class="task-manager-container">  
  @switch(getProductivityLevel()){  
    @case ('excellent'){  
      <div class="alert alert-success">  
        [SUCCESS] Outstanding Productivity!  
        Keep up the great work!  
      </div>  
    }  
    @case ('good'){  
      <div class="alert alert-info">  
        [GOOD] Good Job!  
        You're maintaining steady progress.  
      </div>  
    }  
    @case ('needs-improvement'){  
      <div class="alert alert-warning">  
        [WARNING] Your Productivity could use a boost.  
        Focus on completing tasks!  
      </div>  
    }  
    @case ('poor'){  
      <div class="alert alert-danger">  
        [ALERT] Time to get back on track!  
        Start with small, easy tasks.  
      </div>  
    }  
    @default {  
      <div class="alert alert-info">  
        Keep tracking your tasks!  
      </div>  
    }  
  }  
</div>
```

- Create input text

Add New Task

Title *

HTML:

```
<!-- Add Task section starts -->

<div class="add-task-section">
  <div class="add-task-header">
    <h2>Add New Task</h2>
  </div>

  <div class="form-grid">
    <div class="form-group">
      <label>Title *</label>
      <input type="text" [(ngModel)]="newTask.title" placeholder="Enter task title"
        name="task-title" required (focus)="onFieldFocus('title')" (blur)="onFieldBlur('title')" />
    </div>
  </div>
</div>
```

TS:

```
//Form data
newTask: {
  title: string,
  description: string,
  category: string,
  priority: string,
  dueDate: string | Date,
  status: string
} = {
  title: "",
  description: "",
  category: "",
  priority: 'medium',
  dueDate: "",
  status: 'pending'
};
```

- Category

Category *

Select Category

- ✓ Select Category
- work
- personal
- shopping
- health
- finance
- education
- other

HTML:

```
<div class="form-group">
  <label>Category *</label>
  <select [(ngModel)]="newTask.category" required>
    <option value="" disabled>Select Category</option>
    @for (category of categories; track category)
    {
      <option [value]="category">{{category}}</option>
    }
  </select>
</div>
```

- Priority

Priority

medium

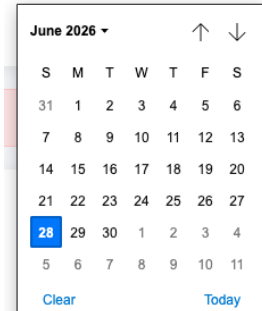
- low
- ✓ medium
- high
- urgent

```
<div class="form-group">
  <label>Priority</label>
  <select [(ngModel)]="newTask.priority">
    @for (priority of priorities; track priority)
    {
      <option [value]="priority">{{priority}}</option>
    }
  </select>
</div>
```

- Due Date

Due Date *

dd/mm/yyyy



```
<div class="form-group">
  <label>Due Date *</label>
  <input type="date" [(ngModel)]="newTask.dueDate" required />
</div>
```

- Status

Status

pending



Status

✓ pending
in-progress
completed
cancelled

```
<div class="form-group">
  <label>Status</label>
  <select [(ngModel)]="newTask.status">
    @for (status of statuses; track status)
    {
      <option [value]="status">{{status}}</option>
    }
  </select>
</div>
```

- Description

Description

Enter task description (optional)



```
<div class="form-group" style="grid-column: 1 / -1;">
  <label>Description</label>
  <textarea [(ngModel)]="newTask.description" placeholder="Enter task description (optional)"></textarea>
</div>
```

- Button disable and clear

Add Task

Clear

```
- <div class="form-actions">
-   <button class="btn btn-primary" (click)="addTask()" [disabled]="!newTask.title || !newTask.category || !newTask.dueDate"
-   [title]="(!newTask.title || !newTask.category || !newTask.dueDate)? 'Please fill in required fields' : 'Add new task'">Add
Task</button>
-
-   <button class="btn btn-secondary" (click)="clearForm()">Clear</button>
- </div>
```

- Alert notification

[ALERT] Time to get back on track! Start with small, easy tasks.

HTML:

```
<!-- Productivity alert starts -->
@switch (getProductivityLevel())
{
  @case ('excellent')
  {
    <div class="alert alert-success">
    [SUCCESS] Outstanding productivity!
    Keep up the great work!
    </div>
  }
  @case ('good')
  {
    <div class="alert alert-info">
    [GOOD] Good job!
    You're maintaining steady progress.
    </div>
  }
  @case ('needs-improvement')
  {
    <div class="alert alert-warning">
    [WARNING] Your productivity could use a boost.
    Focus on completing tasks!
    </div>
  }
  @case ('poor')
  {
    <div class="alert alert-danger">
    [ALERT] Time to get back on track!
    Start with small, easy tasks.
    </div>
  }
  @default
  {
```

```
<div class="alert alert-info">
  Keep tracking your tasks!
</div>
}
}
<!-- Productivity alert ends -->
```

List Item

Tasks

Showing 3 tasks

"Complete Angular Assignment"

"Buy Groceries"

"Team Meeting"

```
<!-- Tasks List starts -->
<div class="tasks-section">
  <div class="section-header">
    <h2>Tasks</h2>
    <span>Showing {{getFilteredTasks().length}} tasks</span>
  </div>
  @if (getFilteredTasks().length > 0){
    <div class="tasks-list">
      @for(task of getFilteredTasks(); track task.id)
      {
        <div class="task-item"
          [class]='priority-' + task.priority
          [class.task-completed]='task.status === 'completed''
        >
          {{task.title | json}}
        </div>
      }
    </div>
  }
</div>
<!-- Tasks List ends -->
```

TOTAL TASKS

3

COMPLETED

1

PENDING

1

OVERDUE

2

COMPLETION RATE

33%

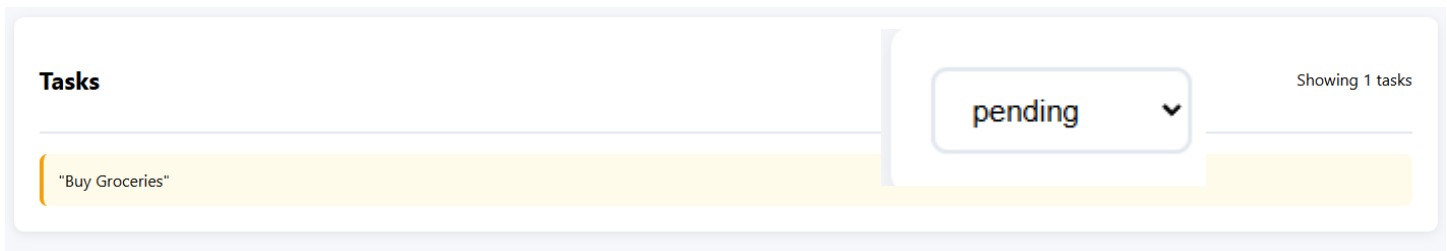
All Status

All Categories

All Priorities

☒ Show Completed Tasks

- This is what we filter based on the drop-down list. If it is pending it will show the list of pending.



HTML:

```
<!-- Tasks List starts -->
<div class="tasks-section">
  <div class="section-header">
    <h2>Tasks</h2>
    <span>Showing {{getFilteredTasks().length}} tasks</span>
  </div>
  @if (getFilteredTasks().length > 0){
    <div class="tasks-list">
      @for(task of getFilteredTasks(); track task.id)
      {
        <div class="task-item"
          [class]=" 'priority-' + task.priority"
          [class.task-completed]="task.status === 'completed'"
        >
          {{task.title | json}}
        </div>
      }
    </div>
  }
</div>
<!-- Tasks List ends -->
```

- This html display base on the condition getFilteredTask()
- We use the dynamic css : [class]=" 'priority-' + task.priority"

```
- /* Priority Colors */
- .priority-low {
-   border-left-color: #10b981 !important;
-   background: #f0fdf4 !important;
- }
-
- .priority-medium {
-   border-left-color: #f59e0b !important;
-   background: #fffbeb !important;
- }
-
- .priority-high {
-   border-left-color: #ef4444 !important;
-   background: #fef2f2 !important;
```

```
- }  
-  
- .priority-urgent {  
-   border-left-color: #dc2626 !important;  
-   background: #fee2e2 !important;  
-   animation: pulse 2s infinite;  
- }  
-
```

- If the task is complete: `[class.task-completed]="task.status === 'completed'"`

It will be used below css style:

Visual summary

Code	What it does
<code>class="task-item"</code>	Always adds the <code>task-item</code> class.
<code>[class]=" 'priority-' + task.priority"</code>	Adds a class like <code>priority-high</code> , <code>priority-medium</code> , or <code>priority-low</code> based on the task's priority.
<code>[class.task-completed]="task.status === 'completed'"</code>	Adds the <code>task-completed</code> class only when the task is completed.

Think of it this way:

- `class="..."` → Always use this class.
- `[class]="..."` → Build the class name dynamically.
- `[class.someClass]="condition"` → Add `someClass` only if the condition is true.

The screenshot shows a web browser window with a task list. The tasks are: "Complete Angular Assignment", "Buy Groceries", and "Team Meeting". The "Team Meeting" task is highlighted in green. Below the browser window, the Chrome DevTools Elements panel is open, showing the HTML structure of the page. The HTML includes a container, a task-section, a section-header, and a tasks-list. The tasks-list contains three task-items: "Complete Angular Assignment" (priority-high), "Buy Groceries" (priority-medium), and "Team Meeting" (priority-high task-completed). The "Team Meeting" task-item is highlighted in green in the Elements panel, matching the highlight in the browser window.

Create list information

1 Complete Angular Assignment

☐ Finish the task manager application with all requirements

education

IN-PROGRESS

Due: 12/15/2024

Delete

- Check box ☐

```
<input type="checkbox" class="task-checkbox" [checked]="task.status === 'completed'"
      (change)="toggleTaskComplete(task.id)"/>
```

- TS:

```
// change status from completed to pending so on
toggleTaskComplete(id: number): void{

  // find the task by id
  const task = this.tasks.find(t => t.id === id);
  if(task){
    if(task.status === 'completed')
    {
      task.status = 'pending';
      delete task.completedAt;
    }
    else
    {
      task.status = 'completed';
      task.completedAt = new Date();
    }
  }
}
```

- បើក នៅក្នុង Condition in the status complete អោយវាប្តូរ ទិន្នន័យនឹងចោលហើយ change status to pending

1 Team Meeting

☒ Discuss Q1 project roadmap

work

COMPLETED

Due: 12/8/2024

Delete

⇒ After we click on the check box it will change to pending

3 Team Meeting

☐ Discuss Q1 project roadmap

work

PENDING

Due: 12/8/2024

Delete

- Show the description and working, complete and due date

```

-         <div class="task-content">
-             <div class="task-header">
-                 <span class="task-number">{{idx + 1}}</span>
-                 <span class="task-title">{{task.title}}</span>
-             </div>
-             @if(task.description)
-             {
-                 <p class="task-description">{{task.description}}</p>
-             }
-             <div class="task-meta">
-                 <span [class]=" 'category-tag category-' +
task.category">{{task.category}}</span>
-                 <span [class]=" 'status-badge status-' +
task.status">{{task.status}}</span>
-                 <span [style.color]="isOverdue(task)? '#ef4444' : '#64748b'">Due:
{{task.dueDate.toLocaleDateString()}}</span>
-             </div>
-         </div>

```

- យើងពិនិត្យមើលលើលខ description បើវាមិនមានទេ យើងមិនចាំបាច់បង្ហាញនោះទេ

```

-         @if(task.description)
-         {
-             <p class="task-description">{{task.description}}</p>
-         }

```

- Condition on the date line: overdue

```

-         isOverdue(task: Task): boolean
-         {
-             const today = new Date();
-             today.setHours(0, 0, 0, 0);
-             return new Date(task.dueDate) < today && task.status != 'completed';
-         }

```

- បើសិនជាថ្ងៃដែលយើងកំណត់ វាផុតកំណត់ តូចជាង ថ្ងៃ នឹង ហើយ status មិនទាន់បានបញ្ចប់ យើងធ្វើការ

⇒ វានឹងធ្វើការ return : true or false (សម្រាប់ធ្វើការកំណត់ style លើលក្ខណ)

```

⇒ <span [style.color]="isOverdue(task)? '#ef4444' : '#64748b'">Due:
{{task.dueDate.toLocaleDateString()}}</span>

```

យើងបន្ទាប់មកយើងបំប្លែងវា ទៅជា String.

- Create Button Click Delete

```

-         <div class="task-actions">
-             <button class="btn btn-danger"
(click)="deleteTask(task.id)">Delete</button>
-         </div>

```

TS:

```
deleteTask(id: number): void
{
  const index = this.tasks.findIndex(task => task.id === id);
  if (index !== -1)
  {
    this.tasks.splice(index, 1);
  }
}
```

⇒ យើងធ្វើការ ស្វែងរក តាមរយៈពេលយើងចុកលើ Button deleted នឹងសិន (click)="deleteTask(task.id) យើងចាប់យក Id ដែលពេលយើងធ្វើការ use event click នឹងសិន

⇒ បើសិនជា id នឹងខុសពី -1 មានន័យថា វាមានតម្លៃ លេខនឹង ចឹងយើងធ្វើការ splice វាចេញ

យើងធ្វើការ ពិនិត្យនឹង @if (getFilteredTasks().length > 0) មានតម្លៃ តែបើសិនវាមិនមានតម្លៃវិញក្នុងលក្ខណ filter

យើងធ្វើការ Filter ទៅលើ ពេលដែលយើង Filter

All Status ▼

work ▼

medium ▼

☒ Show Completed Tasks

Tasks

Showing 0 tasks

[EMPTY]

No tasks match your filters.
Try adjusting your filters or add a new task.

HTML:

```
@else {
  <div class="empty-state">
    <div class="empty-state-icon">[EMPTY]</div>
    <div class="empty-state-title">
      @if(filterStatus !== 'all' || filterCategory !== 'all' || filterPriority !==
'all')
      {
        No tasks match your filters.
      }
      @else {
        No tasks yet.
      }
    </div>
  </div>
}
```

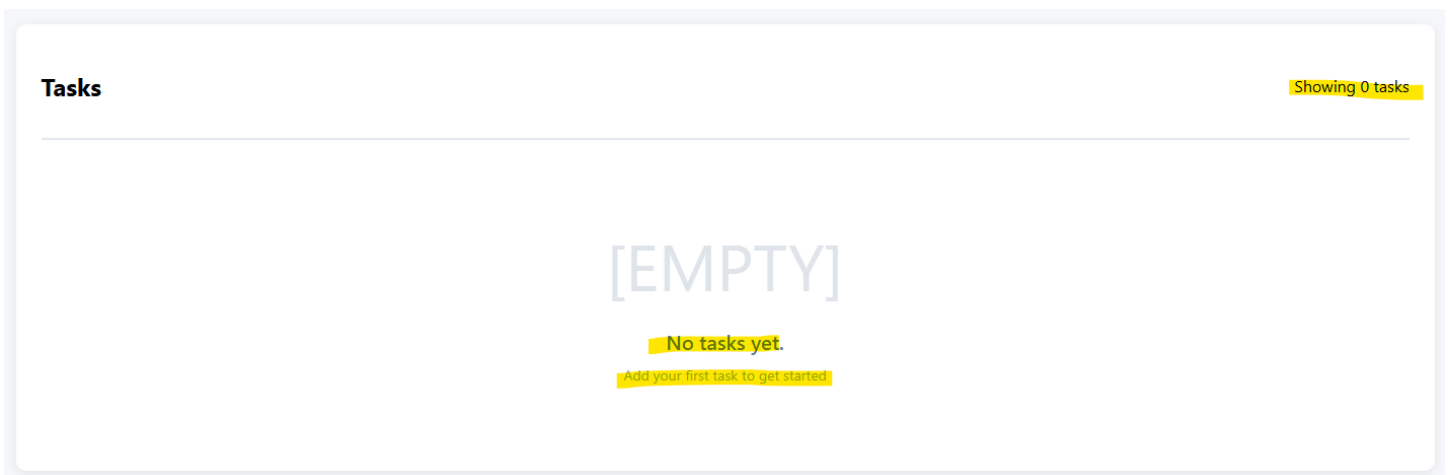
```

    }
  </div>

  <div class="empty-state-description">
    @if(filterStatus !== 'all' || filterCategory !== 'all' || filterPriority !==
'all')
    {
      Try adjusting your filters or add a new task.
    }
    @else {
      Add your first task to get started
    }
  </div>
</div>
}

```

- បើសិនជា filterStatus មិនស្មើ 'all' ... យើងប្រើ else ដើម្បីគ្រាន់តែធ្វើការ កំណត់ style of the text តែប៉ុន្មោះ



បើសិនមិនមាន add task ទេ អោយវាចេញ Add your first task to get started



```

<!-- Productivity Score starts -->
<div class="productivity-score">
  <div class="score-label">Your Productivity Score</div>
  <div class="score-value">{{getCompletionRate()}}%</div>
  <div class="score-label">Keep going!</div>
</div>
<!-- Productivity Score ends -->

```

TaskManager Project Assignment 1 - Part 6

Task Manager - Data Binding Recap

Everything we built in this application

Our Task Manager demonstrates
every Angular data binding
concept

{ }

Interpolation

[]

Property
Binding

()

Event
Binding

↔

Two-Way
Binding

🎨

Style/Class
Binding

@

Structural
Binding

All concepts working together in one real application